

Velora MERN Master

Product requirements, ten-part roadmap and permanent controls

Version 2.0 | 10 October 2026 | Swayam Jain

01 / Authority and product purpose

Version 2.0 - MERN rebuild baseline. Owner: Swayam Jain. Repository:
<https://github.com/Swayamjain1240/Velora>

Verified current repository truth: GitHub reports repository size 0 and no branches or commits. Part 1 will initialize the JavaScript MERN implementation only after inspecting and preserving any local files.

How to use this document

This Master is the product and security authority for the new JavaScript MERN implementation. Read it with VELORA_MERN_ARCHITECTURE and VELORA_MERN_DUCK_LOG before every part. The latest explicit owner decision controls scope, but changes must be recorded in all affected references rather than silently changing code or requirements.

Product purpose

Velora is an AI-assisted post-discharge recovery platform. It turns private discharge sources into clinician-approved recovery plans and later helps patients and authorized caregivers understand approved instructions, record tasks and ask the care team for clarification.

Velora coordinates an existing clinical plan. It does not diagnose, prescribe, decide clinical urgency, certify recovery or promise continuous monitoring. AI produces untrusted drafts only; a qualified assigned human reviewer controls publication.

Locked scope

- Build Code 1 home-recovery companion first. Code 2 diagnostic tests/results is deferred until explicit authorization. Code 3 insurance is not selected.
- Use synthetic identities and documents during development. Real-patient use requires a separate readiness decision.
- Build manual source review and approval before AI prefilling. Provider choices remain open until disclosure and review.
- Target one partner clinic and one procedure/workflow first; both remain unresolved decisions.

02 / Roles and permanent clinical invariants

Role	Allowed responsibility and boundary
Patient	Own profile, episodes, uploads and current approved plan. No clinical approval.
Caregiver	Only explicitly active granular scopes. No automatic document access, onward invitation or approval.
Care coordinator	Assigned operational coordination and manual draft preparation. No clinical publication.
Clinical reviewer	Assigned clinical review/approval and clinical replies; current clinic verification required.
Clinic admin	Staff, roles and settings. Admin alone grants no clinical-record browsing or approval.

Invariants

- Do not guess missing/conflicting dose, route, frequency, timing or duration. Preserve conditions and original source text.
- A new upload or AI result cannot activate instructions, tasks, reminders or patient-visible clinical content.
- Approved content is immutable. A change creates a new reviewed version while the old approved version remains current until replacement approval.
- Self-report, notification delivery, acknowledgement and clinician verification are separate facts.
- Missing medication entries never generate advice to double or replace a dose.
- Clinic-defined response hours and emergency wording remain clear; the app never implies an unstaffed queue is continuously monitored.
- Audit and version history append. Credentials, records and file/model access never cross clinic, ownership, assignment or grant boundaries.

03 / Requirements VEL-001 to VEL-010

ID	Requirement	Target
VEL-001	Account and invitation security	M1 / pilot
VEL-002	Clinic and record isolation	M1
VEL-003	Recovery episodes and assigned team	M1
VEL-004	Private validated document intake	M1
VEL-005	Manual draft and source review	M1
VEL-006	Clinical approval gate	M1
VEL-007	Patient current-approved plan view	M1
VEL-008	AI extraction with source evidence	M2
VEL-009	Reviewed Hindi/English presentation	M2
VEL-010	Immutable care-plan versioning	M1 foundation / later scheduling

Existence of a model, route or page is not acceptance evidence. Each active part must prove the observable workflow, denial paths, concurrency/state behavior and relevant user interface through recorded tests.

04 / Requirements VEL-011 to VEL-021

ID	Requirement	Target
VEL-011	Granular caregiver sharing	Code 1
VEL-012	Medication schedules and actor logs	M3
VEL-013	Daily recovery tasks	M3
VEL-014	Notifications and delivery records	M3 / M4
VEL-015	Clinic-defined recovery check-ins	M4
VEL-016	Clarification queue and history	M4
VEL-017	Assigned team dashboard	M4
VEL-018	Basic prescribed follow-up	M4
VEL-019	Operational closure and history	Code 1
VEL-020	Privacy, audit, export/deletion and retention	Foundation through M5
VEL-021	Workflow metrics without clinical inference	Code 1

Caregiver profile consent does not silently expand to episodes, documents, plans, tasks or questions. Every future scope must be explicit, current, revocable and enforced again on the server and in background jobs.

05 / Ten-part delivery plan

Part	Deliverable	Current state
1	Repository, authentication and clinic permissions	Planned - initialize secure MERN foundation.
2	Patient onboarding, consent and caregiver access	Planned.
3	Recovery episodes and care-team assignment	Planned.
4	Private discharge-document intake	Planned.
5	Manual care plans, review, approval and versioning	Planned - completes synthetic M1 vertical slice.
6	AI extraction, source citations and reviewed language	Planned - M2; provider not selected.
7	Medication schedules, daily tasks and logs	Planned - M3.
8	Check-ins, clarification queue, dashboard and reminders	Planned - M4.
9	Closure, audit, privacy requests, retention and metrics	Planned.
10	Production hardening, restore and clinical readiness	Planned - M5; no deployment authorized by these documents.

Parts are engineering slices. M1-M5 are product milestones. A later planned part is not a defect, but no planned item may be described as implemented or verified.

Core journey

Clinic setup -> verified staff -> patient onboarding/consent -> episode/team -> private source upload -> manual draft -> source review -> eligible reviewer approval -> patient reads current approved version. Later parts add AI draft assistance, tasks/logs, check-ins, clarification, safe reminders and explicit closure.

06 / Security-25 controls 1-13

- 1. Hide API keys and secrets; never expose them in React/VITE variables, logs, errors, Git or screenshots.
- 2. Validate required environment variables at server startup and fail closed with safe messages.
- 3. Protect private React routes and every Express route; the server remains authoritative.
- 4. Use proper authentication: Argon2id/bcrypt passwords, revocable secure cookie sessions, MFA and safe recovery.
- 5. Enforce server-side clinic, role, ownership/assignment/grant and workflow-state access. Default deny.
- 6. Validate and sanitize all bodies, params, queries, headers, forms and uploaded metadata/bytes.
- 7. Prevent XSS and unsafe HTML. Model, document and user text is untrusted data.
- 8. Use appropriate atomic rate, concurrency, body-size and resource limits.
- 9. Use one safe API error format; never expose stack traces, secrets or cross-tenant existence.
- 10. Use an exact credentialed CORS allowlist; no wildcard origin.
- 11. Apply Helmet, explicit CSP and suitable security/file-response headers.
- 12. Production debug mode and verbose diagnostics must be off.
- 13. Check official compatibility and dependency security before upgrades; record unresolved risk.

07 / Security-25 controls 14-25 and viewport rule

- 14. Remove unused packages, dead scaffolding and unnecessary SDKs.
- 15. Check exposed/private files; ignore .env, keys, logs, coverage, builds, private uploads, exports and backups.
- 16. Keep MONGODB_URI and database credentials server-only, TLS/network restricted in production.
- 17. Passwords use Argon2id/bcrypt only; never plaintext or reversible encryption.
- 18. Scan the current Git tree and complete reachable history for leaked secrets.
- 19. Perform and document a security audit before finishing every part.
- 20. After a coherent feature or about 300 meaningful LOC: test, inspect diff, commit and push when authorized. Target 50+ real commits across the project; no fake spam.
- 21. Security is continuous design work, not a final patch.
- 22. Before any external API disclose provider, purpose, API-key name, environment variable and backend/frontend location.
- 23. Test every feature, including denial, invalid input, expiry, revocation, race, retry and failure behavior.
- 24. Keep README, Master, Architecture, Duck Log, API contracts and part reviews synchronized with real code.
- 25. Every authenticated screen fits one viewport without page-level vertical scrolling or clipped content.

One-viewport rule

Never use overflow:hidden to cut required content. Long forms become multi-step wizards; lists and tables use server pagination; many sections use tabs or routes; charts are switchable; mobile shows one primary panel. Test 1440x900, 768x1024, 390x844, 320x568, 844x390 and 320x400. A marketing landing page may intentionally scroll.

08 / Decisions, readiness and change control

MERN decision

The rebuild uses JavaScript only: MongoDB/Mongoose, Express.js, React.js with Vite and Node.js. MongoDB credentials remain in server environment variables. Authentication uses Argon2id and opaque revocable server-side sessions in secure httpOnly cookies; authentication tokens do not belong in localStorage.

Open decisions

Partner clinic, procedure, reviewer verification operations, clinical templates/rules, emergency ownership, hosting, managed MongoDB, private object storage, job queue, AI/OCR provider, notification provider, data region/retention, pilot metrics and pricing remain open.

Before real-patient use

Require clinical workflow review, authorized representative data, privacy/provider review, retention/deletion policy, incident ownership, secure deployment, backup and successful isolated restore, staff onboarding and verified eligibility, extraction/translation evaluation and explicit pilot authorization. CI success is not medical, privacy or regulatory certification.

Change control

Record date, explicit request, affected VEL IDs, rationale, architecture/schema/migration impact and milestone change. Update Master, Architecture, Duck Log, README and applicable API/part review. Never weaken requirements to make existing code appear compliant.

Prepared from the approved ten MERN build prompts on 10 October 2026. No application code was created by this documentation task.